

UD2

FUNDAMENTOS DE PROGRAMACION EN SERVIDOR

MP_0613
Desarrollo web en entorno
servidor

2.1 PHP manual básico
de programación en
servidor.

Antes de empezar: cómo vamos a probar el código

En este manual no vamos a instalar ningún servidor de aplicaciones ni entorno local (XAMPP, WAMP, etc.). Todos los ejemplos y ejercicios se ejecutarán directamente en el navegador usando el editor online gratuito de W3Schools.

- Abre el navegador.
- Entra directamente en: [W3Schools Tryit Editor](https://www.w3schools.com/tryit/)
- Se abrirá el “Tryit Editor”: una pantalla dividida en dos mitades, con el código a la izquierda y el resultado a la derecha.
- Borra el contenido de la izquierda, pega o escribe el código del ejemplo y pulsa el botón verde “Run ►” para ver el resultado.

El propio W3Schools ejecuta el código PHP en su servidor y nos devuelve el HTML resultante, exactamente igual que haría un servidor propio: no necesitamos instalar nada en nuestro equipo.

1. Sintaxis básica

Un archivo PHP es en realidad un archivo HTML normal, con la particularidad de que puede contener bloques de código PHP delimitados por etiquetas especiales. Cuando el servidor procesa el archivo, ejecuta ese código y lo sustituye por el resultado antes de enviarlo al navegador.

```
<?php
    echo "¡Hola, mundo!";
?>
```

- Todo el **código PHP** va entre **<?php** y **?>**.
- Cada **instrucción termina** con **punto y coma (;)**.
- Los **comentarios** se escriben con **//** (una línea) o entre **/*** y ***/** (varias líneas).

PHP **no distingue entre mayúsculas y minúsculas** en los **nombres de funciones e instrucciones** (if, ECHO, Echo... son equivalentes), **pero sí en los nombres de variables**.

2. Directivas: cómo modificar el comportamiento por defecto

Además de sentencias que hacen cosas (asignar un valor, mostrar un mensaje...), PHP admite directivas: instrucciones especiales que no producen ningún resultado visible, sino que cambian la forma en que el propio lenguaje se comporta durante la ejecución del script. La directiva más habitual es `declare(strict_types=1);`, que debe ir en la primera línea del archivo (justo después de `<?php`) y obliga a que los tipos de los parámetros de las funciones se respeten estrictamente, en lugar de que PHP los convierta automáticamente.

```
<?php
declare(strict_types=1);

function doblar(int $numero): int {
    return $numero * 2;
}

echo doblar(4);      // funciona: 8
echo doblar("4");   // error: se esperaba int, no string
```

Sin la directiva `declare(strict_types=1);`, esa misma llamada `doblar("4")` funcionaría sin avisar, porque PHP convertiría automáticamente el texto "4" al número 4. Usar tipos estrictos ayuda a detectar errores antes.

Otras directivas se configuran fuera del propio código, en el archivo de configuración del servidor PHP (`php.ini`) —por ejemplo, el tiempo máximo de ejecución de un script o el tamaño máximo de un archivo subido—, aunque también se pueden ajustar puntualmente desde el propio script con la función `ini_set()`. Como en W3Schools no gestionamos el servidor, no podremos tocar estas últimas, pero conviene saber que existen.

3. Variables y tipos de datos

En PHP las **variables** se declaran con el **símbolo \$ delante del nombre**, y no es necesario indicar su tipo: PHP lo deduce automáticamente a partir del valor asignado.

```
<?php
$nombre = "Ana";      // string
$edad = 21;           // int
$altura = 1.68;       // float
$esAlumna = true;     // bool
$sinValor = null;     // null
```

```
echo $nombre;  
?>
```

Los tipos de datos básicos son:

- **string, int, float, bool:** Cadenas de texto o enteros, decimales y valores lógicos verdadero/falso.
- **array:** Colecciones de valores (las veremos en el apartado de arrays).
- **null:** Ausencia de valor.

Se puede **comprobar el tipo de una variable** con la función **gettype(\$variable)**, muy útil mientras se está aprendiendo.

4. Ámbito de las variables: local y global

El ámbito (o alcance) de una variable determina desde qué partes del script se puede acceder a ella. En PHP, una variable creada dentro de una función es local: solo existe mientras se ejecuta esa función y no es visible desde fuera, aunque fuera exista otra variable con el mismo nombre.

```
<?php  
    $mensaje = "Hola desde fuera";  
  
    function mostrarMensaje() {  
        $mensaje = "Hola desde dentro";  
        echo $mensaje;        // Hola desde dentro  
    }  
  
    mostrarMensaje();  
    echo $mensaje;            // Hola desde fuera (no se ha modificado)  
?>
```

Si de verdad necesitamos que una función use o modifique una variable definida fuera de ella, hay que indicarlo expresamente con la palabra clave global:

```
<?php  
    $contador = 0;  
  
    function incrementar() {  
        global $contador;  
        $contador = $contador + 1;  
    }  
  
    incrementar();  
    incrementar();
```

```
echo $contador;    // 2
?>
```

En general, conviene evitar el uso de variables globales dentro de funciones siempre que sea posible: es preferible pasar los valores como parámetros y devolver el resultado con `return`, tal como veremos en el apartado de funciones. Las variables superglobales como `$_GET` o `$_POST` son una excepción: están disponibles automáticamente en cualquier ámbito, sin necesidad de declararlas como global.

5. Operadores

- **Aritméticos:** `+` `-` `*` `/` `%` (suma, resta, multiplicación, división, resto)
- **De asignación:** `=` `+=` `-=` `*=` `/=` (asignar y modificar a la vez)
- **De comparación:** `==` `===` `!=` `<` `>` `<=` `>=` (el operador `===` compara también el tipo, no solo el valor)
- **Lógicos:** `&&` `||` `!` (Y, O, NO lógico)
- **De concatenación:** `.` (concatena dos cadenas de texto)

```
<?php
$a = 5;
$b = "5";
var_dump($a == $b);    // true  (mismo valor)
var_dump($a === $b);  // false (distinto tipo)
?>
```

6. Salida de datos: echo y concatenación

Para **enviar texto al navegador** se usa **echo** (o **print**, muy similar). Se puede **concatenar texto y variables con el operador punto** (`.`).

```
<?php
$nombre = "Ana";
$edad = 21;
echo "Hola " . $nombre . ", tienes " . $edad . " años.";
?>
```

También se pueden **insertar variables** directamente **dentro** de una **cadena entre comillas dobles**, **sin** necesidad de **concatenar**:

```
<?php
$nombre = "Ana";
```

```
    echo "Hola $nombre, bienvenida.";
?>
```

7. Estructuras de control condicionales

```
<?php
    $nota = 7;

    if ($nota >= 9) {
        echo "Sobresaliente";
    } elseif ($nota >= 5) {
        echo "Aprobado";
    } else {
        echo "Suspenso";
    }
?>
```

Cuando hay muchas comparaciones sobre la misma variable, **switch** resulta más legible que encadenar varios elseif:

```
<?php
    $dia = 3;
    switch ($dia) {
        case 1:
            echo "Lunes";
            break;
        case 2:
            echo "Martes";
            break;
        default:
            echo "Otro día";
    }
?>
```

8. Bucles

- **while:** se repite mientras se cumpla la condición.
- **do...while:** igual que while, pero comprueba la condición al final (se ejecuta al menos una vez).
- **for:** se usa cuando sabemos de antemano cuántas repeticiones necesitamos.
- **foreach:** recorre los elementos de un array.

```
<?php
    for ($i = 1; $i <= 5; $i++) {
```

```
        echo "Número: $i <br>";  
    }  
?>
```

9. Arrays

Un **array** almacena varios valores en una sola variable. Pueden ser **indexados** (por **posición numérica**) o **asociativos** (por **clave de texto**).

```
<?php  
    // Array indexado  
    $frutas = array("manzana", "pera", "plátano");  
    echo $frutas[0];    // manzana  
  
    // Array asociativo  
    $alumno = array("nombre" => "Ana", "edad" => 21);  
    echo $alumno["nombre"];    // Ana  
  
    // Recorrer un array con foreach  
    foreach ($frutas as $fruta) {  
        echo $fruta . "<br>";  
    }  
?>
```

Algunas funciones útiles: **count(\$array)** para saber **cuántos elementos** tiene, y **print_r(\$array)** o **var_dump(\$array)** para **volcar su contenido completo** (útil para depurar).

10. Funciones

Una **función** **agrupa** un **bloque de código reutilizable**. Puede recibir parámetros y devolver un valor con return.

```
<?php  
    function saludar($nombre, $formal = false) {  
        if ($formal) {  
            return "Estimado/a $nombre";  
        }  
        return "¡Hola $nombre!";  
    }  
  
    echo saludar("Ana");  
    echo saludar("Sr. Pérez", true);  
?>
```

En el ejemplo, `$formal` tiene un valor por defecto (`false`), por lo que ese **parámetro** es **opcional** al llamar a la función.

11. Mezclando PHP y HTML

La verdadera utilidad de **PHP** aparece al **combinarlo con HTML** para generar **contenido dinámico**. Se puede entrar y salir del modo PHP tantas veces como haga falta dentro de la misma página.

```
<!DOCTYPE html>
<html>
<body>
  <h1>Lista de la compra</h1>
  <ul>
    <?php
      $productos = array("Leche", "Pan", "Huevos");
      foreach ($productos as $producto) {
        echo "<li>$producto</li>";
      }
    ?>
  </ul>
</body>
</html>
```

Fijémonos en que las etiquetas `<li>` se generan dinámicamente, una por cada elemento del array: si añadimos un producto más al array, aparecerá automáticamente en la lista sin tocar el HTML.

12. Primer contacto con formularios: `$_GET` y `$_POST`

PHP recoge automáticamente los **datos enviados** desde un **formulario HTML** en dos **variables especiales** llamadas **superglobales**: `$_GET` (cuando el formulario usa `method="get"`) y `$_POST` (cuando usa `method="post"`). Profundizaremos en esto más adelante; de momento, un primer ejemplo:

```
<form method="get" action="">
  Nombre: <input type="text" name="nombre">
  <input type="submit" value="Enviar">
</form>

<?php
  if (isset($_GET["nombre"])) {
    echo "Hola " . $_GET["nombre"];
```

```
}  
?>
```

La función `isset()` comprueba si una variable existe (en este caso, si el formulario ya se ha enviado) antes de intentar usarla, evitando errores.

13. Resumen y buenas prácticas

- Usa **nombres de variables descriptivos** (`$precioTotal` en lugar de `$x`).
- **Comenta** el **código** cuando la **lógica no** sea **evidente** a simple vista.
- **Utiliza** `===` en lugar de `==` cuando quieras **comparar también el tipo de dato**.
- **Indent** el **código** dentro de llaves `{}` para que sea legible.
- **Comprueba** siempre **con** `isset()` si una **variable existe** antes de usarla, especialmente con `$_GET` y `$_POST`.